Hibernate Query Language (HQL), JPQL & Native Query Notes

1. Introduction

Hibernate provides HQL (Hibernate Query Language) to query Java entities instead of database tables.
JPQL (Java Persistence Query Language) is the standard query language defined by the JPA specification.
Both HQL and JPQL use entity names and object properties instead of table and column names.

2. HQL (Hibernate Query Language)

HQL is an object-oriented query language used with Hibernate.
Features:
• Works with Entity classes
• Database independent
• Supports CRUD operations
• Supports joins, sorting, grouping and aggregate functions

Example:
FROM Student
FROM Student WHERE age > 18
FROM Student ORDER BY name ASC

3. JPQL (Java Persistence Query Language)

JPQL is the standard query language defined by JPA.
Hibernate fully supports JPQL.
Syntax is almost identical to HQL.

Example:
SELECT s FROM Student s WHERE s.age > 18

4. HQL vs JPQL

HQL:
• Specific to Hibernate
• Supports Hibernate-specific features

JPQL:
• Defined by JPA specification
• Portable across JPA implementations
• Preferred in Spring Boot applications

5. @Query Annotation

Spring Data JPA provides @Query to write custom queries.

Example (JPQL):

@Query("SELECT s FROM Student s WHERE s.name = :name")
List findStudentByName(String name);

Example (HQL style):
@Query("FROM Student WHERE age > :age")
List findOlderStudents(int age);

6. HQL Fetch Keyword

The FETCH keyword loads related entities in a single query.

Example:
SELECT d FROM Department d
JOIN FETCH d.students

Advantages:
• Avoids LazyInitializationException
• Reduces additional SQL queries
• Improves performance

7. Aggregate Functions in HQL

Supported aggregate functions:
COUNT()
SUM()
AVG()
MIN()
MAX()

Examples:
SELECT COUNT(s) FROM Student s
SELECT AVG(s.marks) FROM Student s
SELECT MAX(s.salary) FROM Employee s

8. Native Query

A Native Query is a SQL query written directly for the database.

Advantages:
• Full SQL support
• Database-specific features
• Better for complex SQL

Example:
@Query(value="SELECT * FROM student WHERE age>18",
nativeQuery=true)
List getStudents();

9. nativeQuery Attribute

The nativeQuery attribute tells Spring Data JPA to execute SQL instead of JPQL.

Example:
@Query(value="SELECT * FROM student",
nativeQuery=true)

false (default) → JPQL
true → SQL

10. JPQL vs Native Query

JPQL
• Uses Entity names
• Database independent
• Object oriented

Native Query
• Uses Table names
• Database specific
• Supports all SQL features

11. Repository Example

public interface StudentRepository extends JpaRepository{

@Query("SELECT s FROM Student s WHERE s.age>:age")
List findStudents(int age);

@Query(value="SELECT * FROM student", nativeQuery=true)
List getAllStudents();

}

12. Advantages of @Query

• Custom queries
• Readable code
• Supports JPQL and SQL
• Named parameters
• Complex joins
• Better flexibility

13. Summary

HQL is Hibernate's query language.
JPQL is the JPA standard query language.
Spring Data JPA uses @Query to execute JPQL or Native SQL.
Native queries are useful when JPQL cannot express database-specific SQL.